

# networking & internet data

CS- 3102



Websockets

Updating and Fetching Data

Deleting and Sending data

Sending Data to the Screen

# Websockets in Flutter

- WebSockets enable real-time, two-way communication between the app and the server.
  - Unlike normal HTTP, the connection stays open for continuous data exchange.
  - Both client and server can send and receive messages anytime.
- Common use cases:
    - Chat or messaging apps
    - Online games
    - Live dashboards
    - Notifications

# Connect to a WebSocket Server

- web\_socket\_channel package provides tools to connect
- Creates a WebSocketChannel object
- This object lets us both send and receive messages

```
final channel = WebSocketChannel.connect(  
    Uri.parse('wss://echo.websocket.events'),  
);
```

# Connect to a Websocket Server

- `channel.stream` gives us messages from the server
- `StreamBuilder` rebuilds UI whenever a new message arrives

```
StreamBuilder(  
  stream: channel.stream,  
  builder: (context, snapshot) {  
    return Text(snapshot.hasData ? '${snapshot.data}' : '');  
  },  
);
```



# Listening for Messages

- `channel.stream` gives us messages from the server
- `StreamBuilder` rebuilds UI whenever a new message arrives

```
StreamBuilder(  
  stream: channel.stream,  
  builder: (context, snapshot) {  
    return Text(snapshot.hasData ? '${snapshot.data}' : '');  
  },  
);
```

# Send Data to Server

- Use `sink.add()` to send a message
- Works instantly since the connection is already open

```
_channel.sink.add('Hello!');
```



# Closing the Connection

- Always close the connection when done
- Prevents memory leaks and background activity

```
_channel.sink.close();
```



# Delete and Send Data to the Internet in Flutter

- Modern apps rely on the internet for data. Flutter communicates with online APIs using HTTP requests.

- The main HTTP methods are: GET, POST, PUT, and DELETE.
- We'll focus on POST (send) and DELETE (remove).

# Requirements

1. Add the HTTP dependency in pubspec.yaml
2. Run flutter pub get to install the package.

```
dependencies:  
  http: ^1.5.0
```

```
C:\Users\ \Desktop\flutter_lesson>flutter pub get  
Resolving dependencies... (2.7s)  
Downloading packages... (1.3s)
```

# Import the package



- Import the HTTP package and JSON support at the top of your Dart file.

```
import 'package:http/http.dart' as http;  
import 'dart:convert';
```

# Sending Data (HTTP POST)



- The POST method sends new data to the server. Use `jsonEncode` to send JSON-formatted data.

```
Future<void> sendData() async {  
  final url = Uri.parse('https://example.com/api/users');  
  final response = await http.post(  
    url,  
    headers: {'Content-Type': 'application/json'},  
    body: jsonEncode({'name': 'Mer', 'email': 'mer@example.com'}),  
  );  
  print(response.statusCode == 201 ? 'Sent!' : 'Send Failed!');  
}
```

# Deleting Data (HTTP DELETE)



- The DELETE method removes a record from the server using its ID.

```
Future<void> deleteData() async {  
    final url = Uri.parse('https://example.com/api/users/1');  
    final response = await http.delete(url);  
    print(response.statusCode == 200 ? 'Deleted!' : 'Delete Failed!');  
}
```



# Fetching and Updating in Flutter

# Requirements

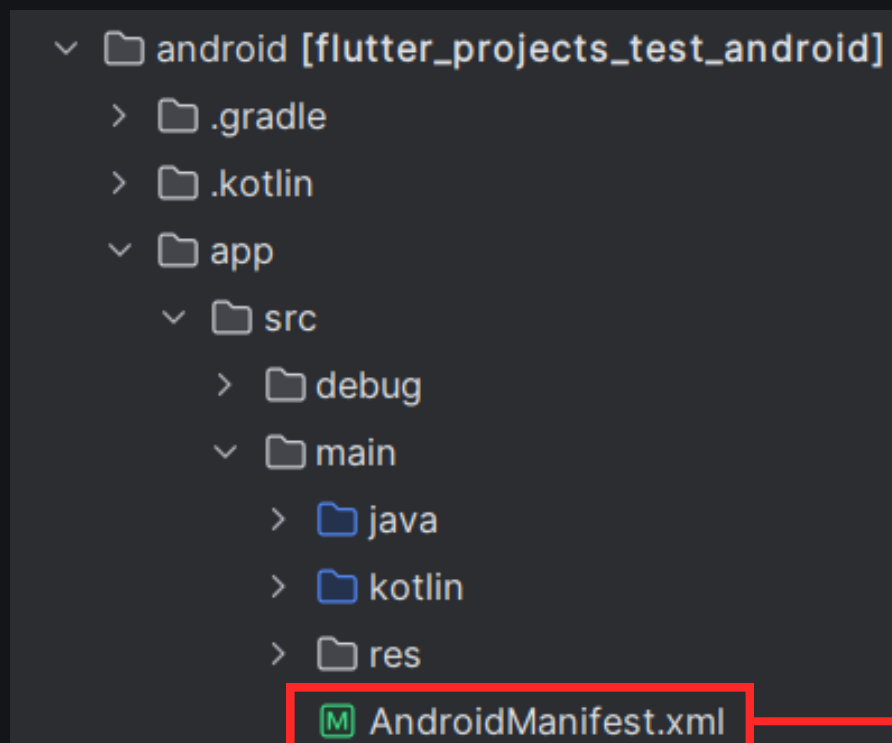
## HTTP Package

```
flutter pub add http
```

## Importing the package

```
import 'package:http/http.dart' as http;  
import 'dart:convert';
```

## AndroidManifest



```
<uses-permission android:name="android.permission.INTERNET"/>
```



# Fetching in Flutter

- Fetching means requesting information from a web API or a backend database
- In terms of CRUD, this is the Read (Fetch)
- When you open your feed, the app fetches the latest posts, likes, and comments from the server.

# Fetching Data (HTTP GET)

```
Future<void> fetchUser() async {  
  final response = await http.get(  
    Uri.parse('https://api.example.com/user/1'),  
  );  
  
  if (response.statusCode == 200) {  
    final data = jsonDecode(response.body);  
    print('User Name: ${data['name']}');  
  } else {  
    print('Failed to fetch data. Error: ${response.statusCode}');  
  }  
}
```

# Updating Data (HTTP PUT / PATCH)

- Updating means sending modified data back to the server
- In terms of CRUD, this is the UPDATE
- When you edit your bio or profile picture, the app updates your data on the server.

# Updating Data (HTTP PUT/PATCH)

```
Future<void> updateUser() async {  
  final response = await http.put(  
    Uri.parse('https://api.example.com/user/1'),  
    headers: {'Content-Type': 'application/json'},  
    body: jsonEncode({  
      'name': 'Amiel',  
      'email': 'amiel@gmail.com',  
    }),  
  );  
}
```

# Sending Data to the Screen

It refers to the process of fetching data from an external source (like an API or database) and displaying it in the user interface.

This involves handling asynchronous operations, managing loading states, and rendering the data using Flutter widgets.



# Two Primary Methods

| Method               | Data Type                       | Updates               | Use Case                                 | Example                                 | Connection                   |
|----------------------|---------------------------------|-----------------------|------------------------------------------|-----------------------------------------|------------------------------|
| <b>FutureBuilder</b> | Single result<br>(Future)       | One-time only         | HTTP<br>requests<br>(GET, POST,<br>etc.) | Fetch user<br>profile, load<br>products | Closes after<br>completion   |
| <b>StreamBuilder</b> | Multiple<br>results<br>(Stream) | Continuous<br>updates | WebSockets,<br>real-time data            | Chat<br>messages, live<br>scores        | Stays open<br>until disposed |

# FutureBuilder Method

FutureBuilder Code Snippet

```
1 class UserListScreen extends StatelessWidget {  
2   @override  
3   Widget build(BuildContext context) {  
4     return Scaffold(  
5       appBar: AppBar(title: Text('Users')),  
6       body: FutureBuilder<List<User>>(  
7         future: fetchUsers(),  
8         builder: (context, snapshot) {  
9  
10          if (snapshot.connectionState ==  
11             ConnectionState.waiting) {  
12            return Center(  
13              child: CircularProgressIndicator()  
14            );  
15          }  
16  
17          if (snapshot.hasError) {  
18            return Center(  
19              child: Text('Error: ${snapshot.error}')  
20            );  
21          }  
22        }  
23      )  
24    );  
25  }  
26 }
```

## Code Breakdown

- **future:** The async function that fetches data
- **builder:** Rebuilds UI based on snapshot state
- **connectionState.waiting:** Shows loading indicator while fetching
- **hasError:** Displays error message if request fails

# FutureBuilder Method

```
22
23     if (snapshot.hasData) {
24         return ListView.builder(
25             itemCount: snapshot.data!.length,
26             itemBuilder: (context, index) {
27                 final user = snapshot.data![index];
28                 return ListTile(
29                     title: Text(user.name),
30                     subtitle: Text(user.email),
31                 );
32             },
33         );
34     }
35
36     return Center(child: Text('No data'));
37 },
38 ),
39 );
40 }
41 }
```

## Code Breakdown

- **hasData:** Renders the fetched data in a list
- **snapshot.data:** Contains the actual fetched data



# StreamBuilder Method

```
StreamBuilder Code Snippet

1 class ChatScreen extends StatefulWidget {
2   @override
3   _ChatScreenState createState() =>
4     _ChatScreenState();
5 }
6
7 class _ChatScreenState extends State<ChatScreen> {
8   final channel = WebSocketChannel.connect(
9     Uri.parse('wss://echo.websocket.org'),
10  );
11
12  @override
13  Widget build(BuildContext context) {
14    return Scaffold(
15      appBar: AppBar(title: Text('Live Chat')),
16      body: StreamBuilder(
17        stream: channel.stream,
18        builder: (context, snapshot) {
19
20          if (snapshot.connectionState ==
21              ConnectionState.waiting) {
22            return Center(
23              child: Text('Connecting...')
24            );
25          }
26        }
27      )
28    );
29  }
```

## Code Breakdown

- **WebSocketChannel.connect:**  
Establishes WebSocket connection
- **stream:** The continuous data stream to listen to
- **builder:** Rebuilds on every new message
- **connectionState.waiting:** Shows connecting message

# StreamBuilder Method

```
27     if (snapshot.hasError) {
28         return Center(
29             child: Text('Error: ${snapshot.error}')
30         );
31     }
32
33     if (snapshot.hasData) {
34         return Center(
35             child: Text('Message: ${snapshot.data}')
36         );
37     }
38
39     return Center(child: Text('No messages'));
40 },
41 ),
42 );
43 }
44
45 @override
46 void dispose() {
47     channel.sink.close();
48     super.dispose();
49 }
50 }
```

## Code Breakdown

- **hasData:** Displays each new message received
- **dispose():** Closes connection when widget is removed
- **hasError:** checks if an error occurred during the WebSocket connection or data stream.

# demo

thank you